

YOLO11n-seg ONNX Inference and Post-processing Improvements

E-waste Battery Extraction Project - Computer Vision

Prepared by

Tuan Thanh Lu (224259805)

Sprint 1 focus: CV-02 and CV-06

CV-07 is referenced only as future interface preparation

1. Purpose and scope

This report documents the changes I made to the initial YOLO11n-seg ONNX Runtime inference script provided by a teammate. The starting script already contained the main ONNX pipeline: image preprocessing, ONNX Runtime inference, detection decoding, non-maximum suppression, mask reconstruction, box rescaling, centroid calculation and a minimum-area-rectangle (MAR) angle. My work focused on the post-processing problems found during local image testing and on making the battery geometry more useful for later integration.

The work in this document mainly supports Sprint 1 tasks CV-02 (YOLO11n-seg ONNX Compatibility Check) and CV-06 (Detection Post-processing). No ROS 2 cv_interfaces messages were created here, so CV-07 is not treated as completed work. The ordered four-corner output is only preparation for that later task.

2. Starting point

The original onnx_inference.py runs the YOLO11n-seg model directly with ONNX Runtime instead of using the Ultralytics .pt pipeline. It records the model input as images with shape (1, 3, 640, 640), and two outputs: output0 for detections and mask coefficients, and output1 for the prototype masks. It also applies RGB conversion, normalisation, letterbox resizing, NMS and mask reconstruction before saving annotated images and CSV metrics.

The supplied run.py was also kept as a reference implementation using the original .pt model through Ultralytics. That script derives a minimum-area rectangle from the segmentation contour and visualises the rotated rectangle, its centre and the long-axis orientation. I used this behaviour as a reference when improving the geometric output in the ONNX version.

3. Main limitations identified in the initial post-processing

- The reconstructed segmentation mask could contain disconnected regions outside the main battery area during testing.
- The displayed bounding box was the original axis-aligned YOLO box. It therefore became loose when the battery was rotated.
- The initial script calculated an MAR angle, but it did not draw the rotated MAR rectangle or return its four corner points.
- The CSV output did not include the MAR centre, four corners, width or height.

4. Changes made to the ONNX pipeline

4.1 Restrict the mask to the detected battery region

After converting the detection box back to the original image coordinates, I added a mask-cropping step. Only mask pixels inside the detected box are kept. This prevents mask pixels well outside the detected battery region from being carried into later geometry calculations.

```
cropped_mask = np.zeros_like(mask_orig)
cropped_mask[y1:y2, x1:x2] = mask_orig[y1:y2, x1:x2]
mask_orig = cropped_mask
```

4.2 Keep only the largest connected mask region

The cropped mask is then processed with OpenCV contours. The largest contour is retained and filled into a new clean mask. This removes smaller disconnected mask islands while preserving the main battery region used for the centroid and MAR calculation.

```
contours, _ = cv2.findContours(mask_orig, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
if contours:
    largest_contour = max(contours, key=cv2.contourArea)
```

```
clean_mask = np.zeros_like(mask_orig)
cv2.drawContours(clean_mask, [largest_contour], -1, 1, thickness=cv2.FILLED)
mask_orig = clean_mask
```

4.3 Use a rotated minimum-area rectangle for the final geometry

The original axis-aligned rectangle is no longer drawn as the main visual box. Instead, the updated code calculates `cv2.minAreaRect()` from the cleaned battery contour, converts it to four corner points with `cv2.boxPoints()`, and draws the rotated rectangle. This follows the visible battery orientation more closely, especially for rotated test images.

4.4 Add long-axis orientation output

The MAR angle is normalised to the long axis. An orientation arrow is drawn through the MAR centre along that axis. The arrow is used as an orientation indicator; its arrowhead does not represent a unique movement direction because the stored orientation is in a 0-180 degree range.

4.5 Define a deterministic order for the four MAR corners

The updated code adds `order_mar_corners()` so the four points are not left in the raw order returned by OpenCV. The first point is the upper-most corner in image coordinates, with the left-most point used if there is a tie. The remaining points then continue clockwise. The ordered points are stored together as `mar_corners`.

4.6 Expand the saved metrics

The updated CSV keeps the original detection information and adds the MAR centre, angle, four ordered corners, width and height. The original axis-aligned box is still retained internally and in the metrics because it is used during mask restriction and remains useful for debugging.

5. Summary of code changes

Area	Initial script	Updated script
Mask post-processing	Reconstructed binary mask used directly.	Mask is restricted to the detection box, then only the largest connected contour is retained.
Displayed box	Axis-aligned YOLO rectangle.	Rotated MAR derived from the cleaned segmentation contour.
Orientation	MAR angle calculated only.	Long-axis angle retained and an orientation arrow is drawn.
Corner points	Not returned.	Four MAR corners are generated and ordered deterministically.
Saved geometry	YOLO box, centroid and MAR angle.	Adds MAR centre, corners, width and height while keeping the original debugging values.

6. Relation to Sprint 1 tasks

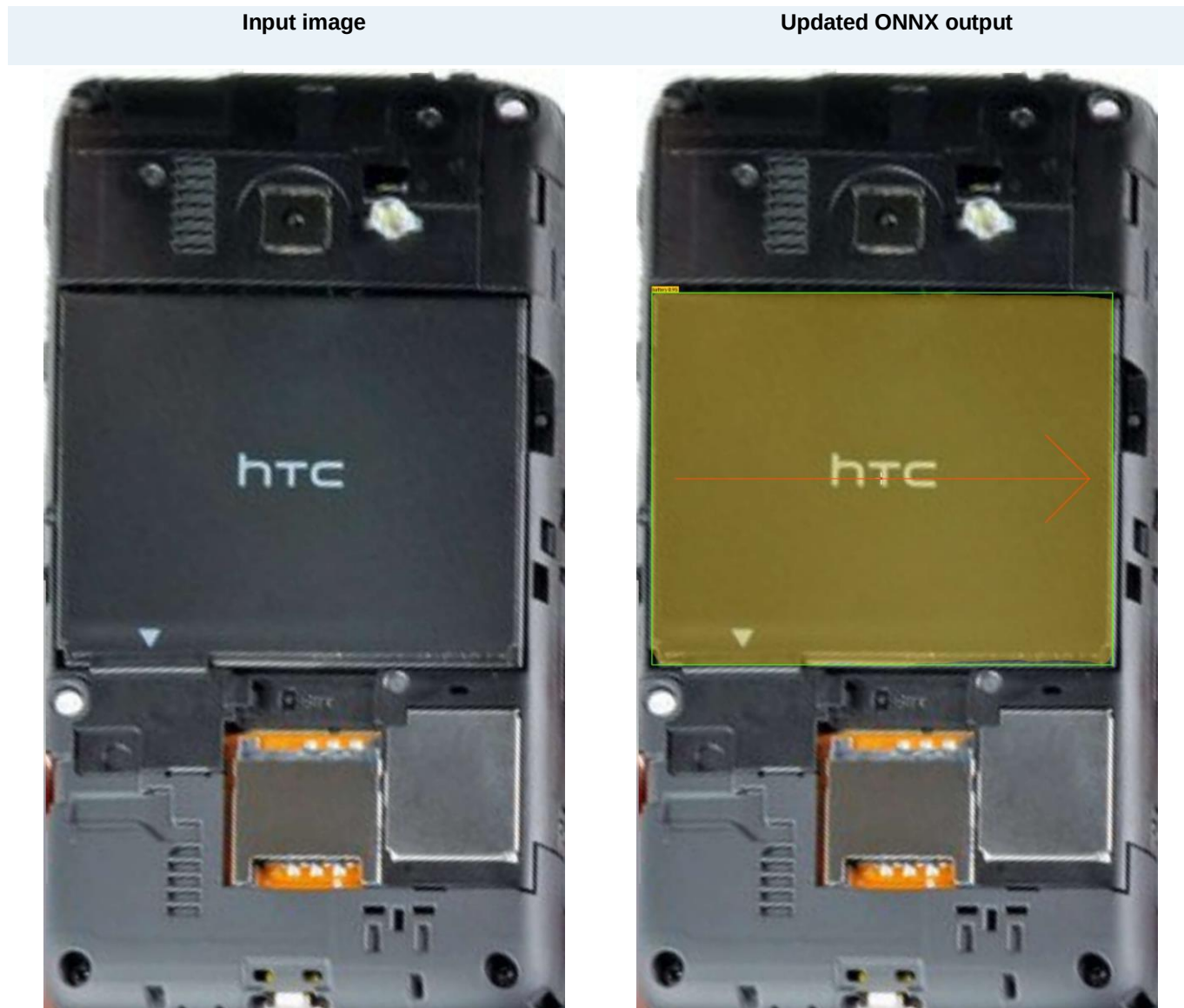
Task	How this work relates	Status in this report
CV-02 - YOLO11n-seg ONNX Compatibility Check	The standalone ONNX Runtime pipeline uses the documented input/output names, tensor shapes and preprocessing and was run locally on saved images.	Supported by this work.
CV-06 - Detection Post-processing	Main focus of the changes: clean the segmentation output and recover more useful battery geometry in original image coordinates.	Primary task addressed.
CV-07 - Create cv_interfaces Messages	The updated script now provides four ordered MAR corners that can later be mapped into a four-point bounding-box message.	Not implemented here; preparation only.
CV-09 - Offline Pipeline Testing	The image pairs below provide local saved-image checks of the updated pipeline.	Only partial evidence; full stability, empty-detection and FPS/latency testing is outside this report.

7. Local input/output examples

The following five pairs show representative saved images that were passed through the updated ONNX pipeline. They are included as visual evidence of the cleaned segmentation, rotated MAR box and orientation output. These examples are not used here as a quantitative accuracy benchmark.

Example 1 - Dark HTC battery

Input: CamScanner_08-09-2025_06.15_103.jpg | Output: CamScanner_08-09-2025_06.15_103(1).jpg



Observation: The updated output detects the dark rectangular battery and produces a tight MAR around the segmented region, with the long-axis indicator drawn horizontally.

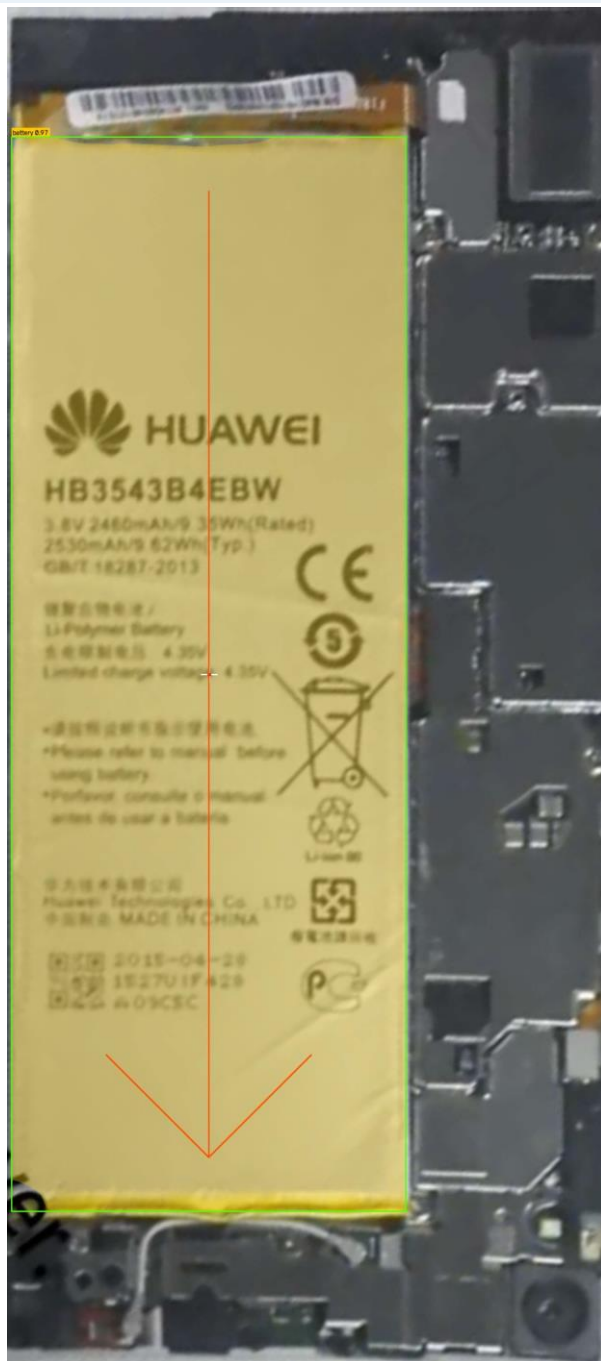
Example 2 - Upright Huawei battery

Input: CamScanner_08-09-2025_06.15_5(5).jpg | Output: CamScanner_08-09-2025_06.15_5(6).jpg

Input image



Updated ONNX output



Observation: For the upright battery, the rotated rectangle is close to axis-aligned and follows the battery edges, while the long-axis orientation is vertical.

Example 3 - Rotated Huawei battery

Input: CamScanner_08-09-2025_06.15_5_aug_9(5).jpeg | Output: CamScanner_08-09-2025_06.15_5_aug_9(6).jpeg

Input image



Updated ONNX output



Observation: The MAR rotates with the battery instead of expanding into a large horizontal/vertical box, which gives a more useful representation of the battery orientation.

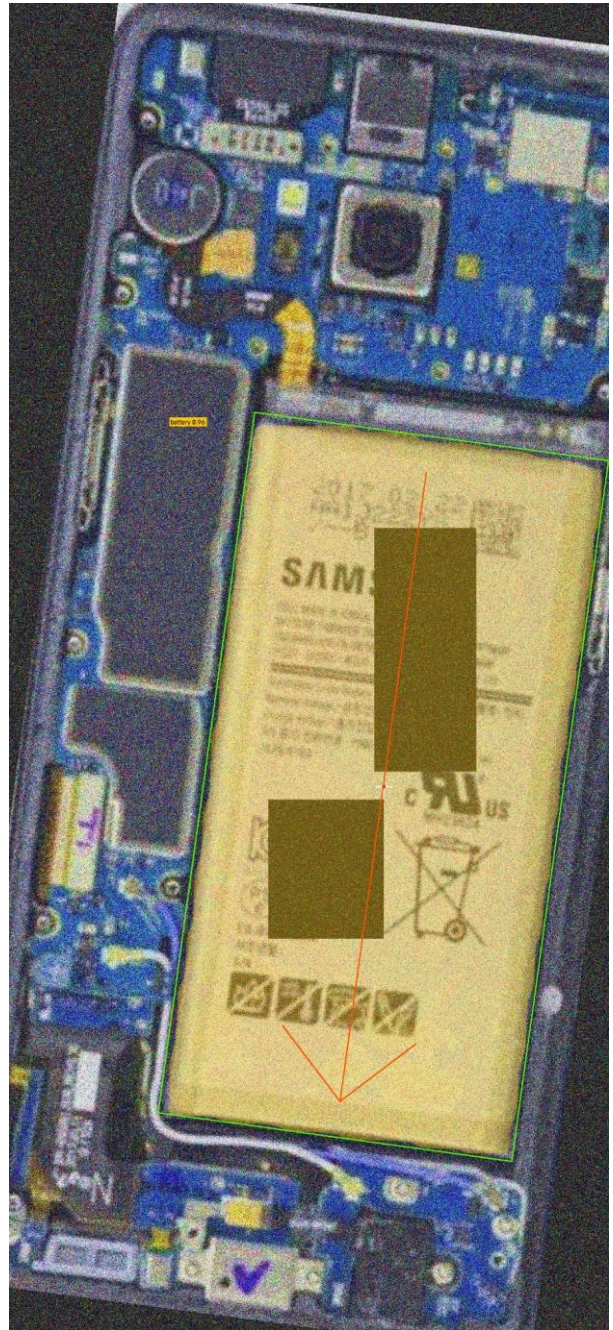
Example 4 - Noisy and occluded Samsung battery

Input: CamScanner_08-09-2025_06.15_7_aug_4(4).jpeg | Output: CamScanner_08-09-2025_06.15_7_aug_4(5).jpeg

Input image



Updated ONNX output



Observation: The battery is still detected in the noisy image with black occlusions. The main segmentation region is retained and the MAR follows the battery orientation.

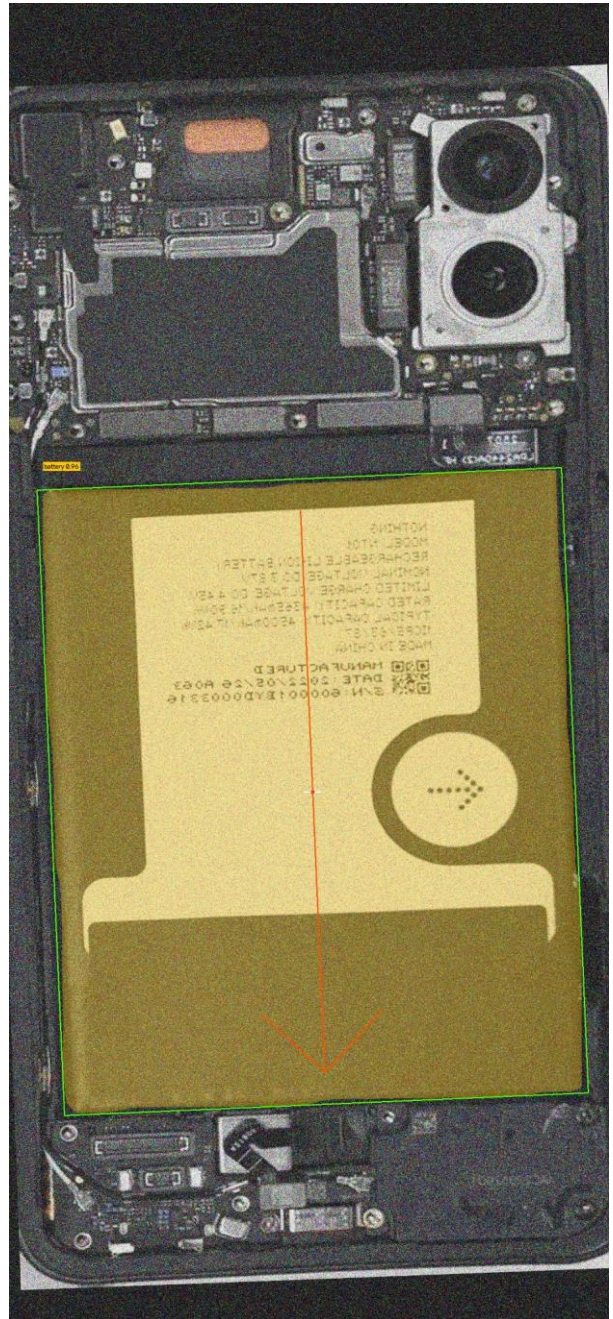
Example 5 - Rotated noisy Nothing battery

Input: CamScanner_08-09-2025_06.15_88_aug_2(4).jpeg | Output: CamScanner_08-09-2025_06.15_88_aug_2(5).jpeg

Input image



Updated ONNX output



Observation: The updated output also handles a different battery shape and appearance, producing a rotated MAR around the main segmented battery region.

8. Outcome and next steps

The updated ONNX script keeps the original direct ONNX Runtime inference pipeline but improves the post-processing around the segmentation output. The main battery mask is restricted and cleaned before geometry is calculated, and the final visual box is now a rotated minimum-area rectangle instead of the axis-aligned YOLO box. The script also exposes the MAR centre, dimensions, angle and four ordered corner points in the saved metrics.

This work remains a standalone local inference and post-processing implementation. It does not create a ROS 2 node or `cv_interfaces` message. The ordered MAR corner output is intended to make that later interface step simpler, but the ROS 2 message definition itself belongs to CV-07 and is outside the scope of this report.

A separate validation document can cover the quantitative comparison between the updated ONNX output and the supplied Ultralytics .pt reference pipeline. Further CV-09 work can also cover empty-detection cases, broader offline testing and basic FPS/latency before the pipeline is wrapped into ROS 2.